

HOWTO submit jobs

From HPC

Contents

- 1 Jobs
- 2 Submitting jobs
 - 2.1 Editing files
 - 2.1.1 Native Editors
 - 2.1.2 External Editors
 - 2.2 Specifying job parameters
 - 2.3 Useful PBS parameters
 - 2.4 Interactive jobs
 - 2.5 Jobs with large output files
 - 2.6 Running OpenMP multi-threaded programs
 - 2.7 Using the PBS_NODEFILE for multi-threaded jobs
 - 2.8 Selecting different nodes in one job
- 3 Guidelines / Rules
- 4 Examples
 - 4.1 ADF
 - 4.2 ANSYS
 - 4.2.1 Fluent
 - 4.2.2 Fluid-Structure Interaction
 - 4.2.3 CFX
 - 4.3 Abaqus
 - 4.4 R
 - 4.5 CPMD
 - 4.6 Gaussian
 - 4.7 pisoFOAM
 - 4.8 MSC Marc
 - 4.9 mothur
 - 4.10 Altair FEKO
 - 4.11 Numeca
- 5 Programs that handle job submission differently
 - 5.1 MATLAB
 - 5.2 GPU

Jobs

The HPC implements a batch queueing system called PBS. Any work you want to do needs to be packaged as a job that the system can do without any interaction from you. Because it's a queue, and the queue might be long, your job might only start a couple of hours from now. It's important to have your script completely self-contained so that it can run without supervision, otherwise, it might just error and all that time waiting in the queue will be wasted.

Submitting jobs

PBS comes with very complete man pages. Therefore, for complete documentation of PBS commands you are encouraged to type `man pbs` and go from there. Jobs are submitted using the `qsub` command. Type `man qsub` for information on the plethora of options that it offers.

Let's say I have an executable called "myprog". Let me try and submit it to PBS:

```
[username@hpc1 ~]$ qsub myprog
qsub: file must be an ascii script
```

Oops... That didn't work because qsub expects a shell script. Any shell should work, so use your favorite one. So I write a simple script called "myscript.sh"

```
#!/bin/bash
cd $PBS_O_WORKDIR
./myprog argument1 argument2
```

and then I submit it:

```
[username@hpc1 ~]$ qsub myscript.sh
qsub: Job has no walltime requested. Request walltime with '-l walltime=HH:MM:SS'.
```

PBS wants to know how long every job is expected to run. This is so that it can schedule jobs optimally. Updating the script to include the PBS directive '-l walltime', we get the script

```
#!/bin/bash
#PBS -l walltime=00:05:00
cd $PBS_O_WORKDIR
./myprog argument1 argument2
```

which we then submit:

```
[username@hpc1 ~]$ qsub myscript.sh
16.hpc1.hpc
```

That worked! Note the use of the \$PBS_O_WORKDIR environment variable. This is important since by default PBS on our cluster will start executing the commands in your shell script from your home directory. To go to the directory in which you executed qsub, cd to \$PBS_O_WORKDIR. There are several other useful PBS environment variables that we will encounter later.

In this script, we also informed PBS that the job will be running for a maximum of 5 minutes.

Editing files

Editing files on the cluster can be done through a couple of different methods...

Native Editors

- vim - The visual editor (vi) is the traditional Unix editor. However, it is not necessarily the most intuitive editor. That being the case, if you are unfamiliar with it, there is a vi tutorial, vimtutor.
- pico - While pico is not installed on the system, nano is installed, and is a pico work-a-like.
- nano - Nano has a good bit of on-screen help to make it easier to use.

External Editors

You can also use your favourite editor on your local machine and then transfer the files over to the HPC afterwards. One caveat to this is that files created on Windows machines usually contain unprintable characters which may be misinterpreted by Linux command interpreters (shells). If this happens, there is a utility called dos2unix that you can use to convert the text file from DOS/Windows formatting to Linux formatting.

```
$ dos2unix script.sub
dos2unix: converting file script.sub to UNIX format ...
```

If you're using MobaXterm's default text editor, make sure your file is in the correct format by selecting UNIX in the Format menu, or checking that the penguin icon in the button bar is selected.

Specifying job parameters

By default, any script you submit will run on a single processor with 1GB of memory. The name of the job will be the name of the script, and it will not email you when it starts, finishes, or is interrupted. stdout and stderr are collected into separate files named after the job number. You can affect the default behaviour of PBS by passing it parameters. These parameters can be specified on the command line or inside the shell script itself. For example, let's say I want to send stdout and stderr to a file that is different from the default:

```
[username@hpc1 ~]$ qsub -e myprog.err -o myprog.out myscript.sh
```

Alternatively, I can actually edit myscript.sh to include these parameters. I can specify any PBS command line parameter I want in a line that begins with "#PBS":

```
#!/bin/bash
#PBS -l walltime=00:05:00
#PBS -e myprog.err
#PBS -o myprog.out
cd $PBS_O_WORKDIR
./myprog argument1 argument2
```

Now I just submit my modified script with no command-line arguments

```
[username@hpc1 ~]$ qsub myscript.sh
```

Useful PBS parameters

Here is an example of a more involved script that requests only 1 hour of execution time, renames the job to 'My-Program', and sends email when the job begins, ends, or aborts:

```
#!/bin/bash
# Name of my job:
#PBS -N My-Program

# Run for 1 hour:
#PBS -l walltime=1:00:00

# Where to write stderr:
#PBS -e myprog.err

# Where to write stdout:
#PBS -o myprog.out

# Send me email when my job aborts, begins, or ends
#PBS -m abe

# This command switched to the directory from which the "qsub" command was run:
cd $PBS_O_WORKDIR

# Now run my program
./myprog argument1 argument2

echo Done!
```

Some more useful PBS parameters:

- -M: Specify your email address (defaults to campus email).
- -j oe: merge standard output and standard error into standard output file.
- -V: export all your environment variables to the batch job.
- -I: run an interactive job (see below).

Once again, you are encouraged to consult the qsub manpage for more options.

Interactive jobs

Normally when you call qsub PBS grabs your script and goes away. You get the prompt returned to you almost immediately and whatever output is produced from your job goes into a designated file. But what if you want to run an interactive job, say to compile something? This is not, in general, the right way to use an HPC, because interactive jobs waste CPU time, mostly because humans are so slow.

```
[username@hpc1 ~]$ qsub -I -l walltime=2:00:00
qsub: waiting for job 1695.hpc1.hpc to start
qsub: job 1695.hpc1.hpc ready

[username@comp007 ~]$ logout

qsub: job 1695.hpc1.hpc completed
[username@hpc1 ~]$
```

In the example above I requested an interactive job, using the defaults of 1 core and 1GB memory and requesting it for 2 hours. If the system can honour the request, I'm immediately given an SSH session to the compute node assigned to my job. Closing the SSH session ends the job, and I'm returned to the login node.

You can also use the qsubi script, which will request an interactive session for 24 hours on a node with 4 cores and 14GB RAM.

```
[username@hpc1 ~]$ qsubi
Requesting interactive session with 4 cores, 14GB RAM, for 24 hours
qsub: waiting for job 1696.hpc1.hpc to start
qsub: job 1696.hpc1.hpc ready

[username@comp008 ~]$ logout

qsub: job 1696.hpc1.hpc completed
[username@hpc1 ~]$
```

Jobs with large output files

If your job makes frequent writes to disk, it may benefit from using scratch space instead of working directly from the home directory.

Instead of a job submission like this:

```
#!/bin/bash
#PBS -N massiveJob
#PBS -l walltime=01:00:00

cd ${PBS_O_WORKDIR}
myprogram -i /home/username/inputfile -o /home/username/outputfile
```

change it to something like this:

```
#!/bin/bash
#PBS -l select=1:ncpus=1
#PBS -l walltime=1:00:00
#PBS -N massiveJob

# make sure I'm the only one that can read my output
umask 0077
# create a temporary directory with the job ID as name in /scratch-small-local
TMP=/scratch-small-local/${PBS_JOBID}
mkdir -p ${TMP}
echo "Temporary work dir: ${TMP}"

# copy the input files to ${TMP}
echo "Copying from ${PBS_O_WORKDIR}/ to ${TMP}/"
/usr/bin/rsync -vax "${PBS_O_WORKDIR}/" ${TMP}/

cd ${TMP}
```


If you have an application that manually forks processes onto the nodes of your job, you are responsible for parsing the PBS_NODEFILE to determine which nodes those are.

Some MPI implementations require you to feed the PBS_NODEFILE to mpirun, e.g. for OpenMPI one may pass -hostfile \${PBS_NODEFILE}.

Be sure to include the mpirun parameter in your resource request. The default is 1, and will cause the PBS_NODEFILE to contain only one line per node if not specified correctly.

Selecting different nodes in one job

Using the above information, one may allocate multiple nodes of the same type, e.g. multiple 48-core nodes. In order to mix multiple different resources, one may use the PBS' "+" notation. For example in order to mix one 48-core node and two 8-core nodes in one PBS job, one may pass:

```
[username@hpc1 ~]$ qsub -l select=1:ncpus=48:mpiprocs=48+2:ncpus=8:mpiprocs=8 -l walltime=1:00:00 myscript.sh
```

Guidelines / Rules

- Create a temporary working directory in **/scratch-small-local** or **/scratch-large-network**, not **/tmp**
 - **/tmp** is reserved for use by the operating system, and is only 5GB in size.
 - Preferably specify **/scratch-small-local/\${PBS_JOBID}** in your submit script so that it's easy to associate scratch directories with their jobs.
 - Copy your input files to your scratch space and work on the data there. Avoid using your home directory as much as possible.
 - Copy only your results back to your home directory. Input files that haven't changed don't need to be copied.
 - Erase your temporary working directory when you're done.
- Secure your work from accidental deletion or contamination by disallowing other users access to your scratch directories
 - `umask 0077` disallows access by all other users on newly created files and directories

Examples

ADF

ADF generates run files which are scripts which contain your data. Make sure to convert it to a UNIX file first using **dos2unix**, and remember to make it executable with **chmod +x**.

ADF script requesting 4 cores, on 1 node, -m selects to mail **begin** and **end** messages and -M is the email address to send to. Requests 1 week walltime.

```
#!/bin/bash
#PBS -N JobName
#PBS -l select=1:ncpus=4
#PBS -l walltime=168:00:00
#PBS -m be
#PBS -M username@sun.ac.za

INPUT=inputfile.run

# make sure I'm the only one that can read my output
umask 0077
TMP=/scratch-small-local/${PBS_JOBID}
mkdir -p ${TMP}

if [ ! -d "${TMP}" ]; then
    echo "Cannot create temporary directory. Disk probably full."
    exit 1
fi
```

```
cd ${TMP}

module load app/adf/2019.103

# override ADF's scratch directory
export SCM_TMPDIR=${TMP}

# override log file
export SCM_LOGFILE="${TMP}/${PBS_JOBID}.logfile"

# Submit job
${PBS_O_WORKDIR}/${INPUT}

# job done, copy everything back
echo "Copying from ${TMP}/ to ${PBS_O_WORKDIR}/"
/usr/bin/rsync -vax ${TMP}/ "${PBS_O_WORKDIR}/"

# delete my temporary files
[ $? -eq 0 ] && /bin/rm -rf ${TMP}
```

ANSYS

Fluent

Fluent script requesting 4 cores, on 1 node, -m selects to mail **begin** and **end** messages and -M is the email address to send to. Requests 1 week walltime.

```
#!/bin/bash
#PBS -N JobName
#PBS -l select=1:ncpus=4:mpiprocs=4:mem=16GB
#PBS -l license_fluent=4
#PBS -l walltime=168:00:00
#PBS -m be
#PBS -e output.err
#PBS -o output.out
#PBS -M username@sun.ac.za

INPUT=inputfile.jou

# make sure I'm the only one that can read my output
umask 0077
TMP=/scratch-small-local/${PBS_JOBID}
mkdir -p ${TMP}

if [ ! -d "${TMP}" ]; then
    echo "Cannot create temporary directory. Disk probably full."
    exit 1
fi

# copy the input files to ${TMP}
echo "Copying from ${PBS_O_WORKDIR}/ to ${TMP}/"
/usr/bin/rsync -vax "${PBS_O_WORKDIR}/" ${TMP}/

cd ${TMP}

# choose version of FLUENT
# Use module avail to see which versions of ansys are available
module load app/ansys/23.2

# Automatically calculate the number of processors
np=$(wc -l < ${PBS_NODEFILE})

fluent 3d -pdefault -cnf=${PBS_NODEFILE} -mpi=intel -g -t${np} -ssh -i ${INPUT}

# job done, copy everything back
echo "Copying from ${TMP}/ to ${PBS_O_WORKDIR}/"
/usr/bin/rsync -vax ${TMP}/ "${PBS_O_WORKDIR}/"

# delete my temporary files
[ $? -eq 0 ] && /bin/rm -rf ${TMP}
```

MPI example using 3 nodes and 3 x 18 cores.

```
#!/bin/bash

#PBS -l select=3:ncpus=18:mpiprocs=18:mem=28GB
```

```

##PBS -l license_fluent=54
##PBS -l walltime=24:00:00
##PBS -l place=scatter
##PBS -N 46krpm-grad

##PBS -e fluent.err
##PBS -o fluent.out

INPUT=/home/gerhardv/46krpm-grad/journal-1.jou
TMP=/scratch2
mkdir -p ${TMP}

if [ ! -d "${TMP}" ]; then
    echo "Cannot create temporary directory. Disk probably full."
    exit 1
fi

# copy the input files to ${TMP}
echo "Copying from ${PBS_O_WORKDIR}/ to ${TMP}/"
/usr/bin/rsync -vax "${PBS_O_WORKDIR}"/ ${TMP}/

cd ${TMP}

module load app/ansys/23.2
module load intel/mpi

np=$(wc -l < ${PBS_NODEFILE})
fluent 3ddp -pdefault -cnf=${PBS_NODEFILE} -mpi=intel -g -t${np} -ssh -i ${INPUT}

# job done, copy everything back
echo "Copying from ${TMP}/ to ${PBS_O_WORKDIR}/"
/usr/bin/rsync -vax ${TMP}/ "${PBS_O_WORKDIR}/"

# delete my temporary files
[ $? -eq 0 ] && /bin/rm -rf ${TMP}

```

Fluid-Structure Interaction

You need the following 5 files:

- coupling (.sci) file
- structural data (.dat) file
- case (.cas.gz) file
- journal (.jnl) file
- submit script (.sh)

The coupling file should contain two participants. The names of these participants should not have spaces in them. In the example below, Solution 4 should be renamed to something like Solution4. Make sure to replace all instances of the name in the file.

```

<SystemCoupling Ver="1">
  <Participants Count="2">
    <Participant Ver="1" Type="0">
      <Name PropType="string">Solution 4</Name>
      <DisplayName PropType="string">0012 V2</DisplayName>
      <SupportsCouplingIterations PropType="bool">True</SupportsCouplingIterations>
      <UnitSystem PropType="string">MKS_STANDARD</UnitSystem>
      <Regions Count="1">
<--- snip --->

```

The journal file should contain (replace the filename on the ‘rc’ line with your case file):

```

file/start-transcript Solution.trn
file set-batch-options , yes ,
rc FFF-1.1-1-00047.cas.gz
solve/initialize/initialize-flow
(sc-solve)
wcd FluentRestart.cas.gz
exit
ok

```

The job script is given below. Update the COUPLING, STRUCTURALDATA, JOURNAL and NPA variables to reflect your case.

```

#!/bin/bash

#PBS -N fsi
#PBS -l select=1:ncpus=48:mpiprocs=48:mem=90GB
#PBS -l license_fluent=48
#PBS -l walltime=24:00:00

COUPLING=coupling.sci
STRUCTURALDATA=ds.dat
JOURNAL=fluent.journal

# number of processors for Ansys
NPA=8
# Automatically calculate the number of processors left over for Fluent
NP=$(wc -l < ${PBS_NODEFILE})
NPF=$((NP-NPA))

# make sure I'm the only one that can read my output
umask 0077

# create a temporary directory with a random name in /scratch-small-local
TMP=/scratch-small-local/${PBS_JOBID}
mkdir -p ${TMP}
echo "Temporary work dir: ${TMP}"

if [ ! -d "${TMP}" ]; then
    echo "Cannot create temporary directory. Disk probably full."
    exit 1
fi

# copy the input files to ${TMP}
echo "Copying from ${PBS_O_WORKDIR}/ to ${TMP}/"
/usr/bin/rsync -vax "${PBS_O_WORKDIR}/" ${TMP}/

cd ${TMP}

module load app/ansys/20.1

# Start coupling program
/apps/ansys_inc/v162/aisol/.workbench -cmd ansys.services.systemcoupling.exe -inputFile ${COUPLING} &

# Wait until scServer.scs is created
TIMEOUT=60
while [ ! -f scServer.scs -a $TIMEOUT -gt 0 ]; do
    TIMEOUT=$((TIMEOUT-1))
    sleep 2
done

if [ -f scServer.scs ]; then

    # Parse the data in scServer.scs
    readarray JOB < scServer.scs
    HOSTPORT=(${JOB[0]//@/ })

    # Run Fluent
    fluent 3ddp -g -t${NPF} -driver null -ssh -sctest=${HOSTPORT[0]} -sctest=${HOSTPORT[1]} -sctest=${JOB[4]} < ${JOURNAL} >
    output.FLUENT &

    # Run Ansys
    ansys162 -b -sctest=${HOSTPORT[0]} -sctest=${HOSTPORT[1]} -sctest=${JOB[2]} -i ${STRUCTURALDATA} -o output.ANSYS -np ${NPA}

    # job done, copy everything back
    echo "Copying from ${TMP}/ to ${PBS_O_WORKDIR}/"
    /usr/bin/rsync -vax ${TMP}/ "${PBS_O_WORKDIR}/"

fi

# delete my temporary files
[ $? -eq 0 ] && /bin/rm -rf ${TMP}

```

CFX

CFX script requesting 4 cores, on 1 node, -m selects to mail begin and end messages and -M is the email address to send to. Requests 1 week walltime.

```

#!/bin/bash
#PBS -N JobName
#PBS -l select=1:ncpus=4:mpiprocs=4
#PBS -l walltime=168:00:00
#PBS -m be

```

```

#PBS -e output.err
#PBS -o output.out
#PBS -M username@sun.ac.za

DEF=inputfile.def
INI=inputfile.ini

# make sure I'm the only one that can read my output
umask 0077
TMP=/scratch-small-local/${PBS_JOBID}
mkdir -p ${TMP}

if [ ! -d "${TMP}" ]; then
    echo "Cannot create temporary directory. Disk probably full."
    exit 1
fi

# copy the input files to ${TMP}
echo "Copying from ${PBS_O_WORKDIR}/ to ${TMP}/"
/usr/bin/rsync -vax "${PBS_O_WORKDIR}"/ ${TMP}/

cd ${TMP}

module load app/ansys/16.2

# get list of processors
PAR=$(sed -e '[:q;N;s/\n/,/g;t q]' ${PBS_NODEFILE})

cfx5solve -def ${DEF} -ini ${INI} -par-dist ${PAR}

# job done, copy everything back
echo "Copying from ${TMP}/ to ${PBS_O_WORKDIR}/"
/usr/bin/rsync -vax ${TMP}/ "${PBS_O_WORKDIR}/"

# delete my temporary files
[ $? -eq 0 ] && /bin/rm -rf ${TMP}

```

Abaqus

Abaqus script requesting 4 cores, on 1 node, -m selects to mail begin and end messages and -M is the email address to send to.

```

#!/bin/bash
#PBS -l select=1:ncpus=4:mpiprocs=4
#PBS -l walltime=1:00:00
#PBS -m be
#PBS -M username@sun.ac.za

# the input file without the .inp extension
JOBNAME=xyz

# make sure I'm the only one that can read my output
umask 0077
TMP=/scratch-small-local/${PBS_JOBID}
mkdir -p ${TMP}

if [ ! -d "${TMP}" ]; then
    echo "Cannot create temporary directory. Disk probably full."
    exit 1
fi

# copy the input files to ${TMP}
echo "Copying from ${PBS_O_WORKDIR}/ to ${TMP}/"
/usr/bin/rsync -vax "${PBS_O_WORKDIR}"/ ${TMP}/

cd ${TMP}

module load app/abaqus21

# Automatically calculate the number of processors
np=$(wc -l < ${PBS_NODEFILE})

abaqus job=${JOBNAME} input=${JOBNAME}.inp analysis cpus=${np} scratch=${TMP} interactive
wait

# job done, copy everything back
echo "Copying from ${TMP}/ to ${PBS_O_WORKDIR}/"
/usr/bin/rsync -vax ${TMP}/ "${PBS_O_WORKDIR}/"

```

```
# delete my temporary files
[ $? -eq 0 ] && /bin/rm -rf ${TMP}
```

R

R script requesting 1 node, -m selects to mail **begin** and **end** messages and -M is the email address to send to.

1. Using R CMD BATCH

```
#!/bin/bash

#PBS -l select=1:ncpus=2:mem=4GB
#PBS -l walltime=00:30:00
#PBS -M username@sun.ac.za
#PBS -m be

cd ${PBS_O_WORKDIR}

module load app/R/4.5.1

R CMD BATCH script.R
```

2. Just writing everything in the bash script

```
#!/bin/bash

#PBS -N compPredD8
#PBS -l nodes=1:ppn=8
#PBS -l mem=120GB
#PBS -l walltime=2:00:00
#PBS -M username@sun.ac.za
#PBS -m abe

cd ${PBS_O_WORKDIR}

module load app/R/4.5.1

R --vanilla <<RSCRIPT
{
library(parallel)
library(doSNOW)

load("/home/nienkemperj/dataset1.rdat")
load("/username/GPAmadF/dataset2.rdat")
load("/home/username/dataset.rdat3")
load("/home/username/dataset.rdat4")

THE REST OF THE R CODE

}
RSCRIPT
```

3. Via a interactive session

```
qsub -I -l walltime=2:00:00 -l select=1:ncpus=2:mem=6GB
module load app/R/4.5.1
R
```

then you can play around with small code bits to learn stuff etc.

CPMD

CPMD script requesting 8 cores on 1 node, -N names the job 'cpmd', -m selects to mail **end** message and -M is the email address to send to. CPMD runs with MPI which needs to be told which nodes it may use. The list of nodes it may use is given in \$PBS_NODEFILE.

```
#!/bin/bash
#PBS -N cpmd
```

```

#PBS -l select=1:ncpus=8:mpiprocs=8
#PBS -l walltime=1:00:00
#PBS -m e
#PBS -M username@sun.ac.za

module load compilers/gcc-4.8.2
module load openmpi-x86_64

cd ${PBS_O_WORKDIR}

# Automatically calculate the number of processors
np=$(wc -l < ${PBS_NODEFILE})

mpirun -np ${np} --hostfile ${PBS_NODEFILE} /apps/CPMD/3.17.1/cpmd.x xyz.inp > xyz.out

```

Gaussian

Gaussian has massive temporary files (.rwf file). Generally we don't care about this file afterward, so this script doesn't copy it from temporary storage after job completion. Requests 6 week walltime.

```

#!/bin/bash
#PBS -N SomeHecticallyChemicalName
#PBS -l select=1:ncpus=8:mpiprocs=8:mem=16GB:scratch=true
#PBS -l walltime=1008:00:00
#PBS -m be

INPUT=input.cor

# make sure I'm the only one that can read my output
umask 0077
TMP=/scratch-small-local/${PBS_JOBID}
TMP2=/scratch-large-network/${PBS_JOBID}
mkdir -p ${TMP} ${TMP2}

if [ ! -d "${TMP}" ]; then
    echo "Cannot create temporary directory. Disk probably full."
    exit 1
fi

if [ ! -d "${TMP2}" ]; then
    echo "Cannot create temporary directory. Disk probably full."
    exit 1
fi

export GAUSS_SCRDIR=${TMP}

# copy the input files to ${TMP}
echo "Copying from ${PBS_O_WORKDIR}/ to ${TMP}/"
/usr/bin/rsync -vax "${PBS_O_WORKDIR}/" ${TMP}/

cd ${TMP}

# make sure input file has %RWF line for specifying temporary storage
if [ -z "$(/bin/grep ^%RWF ${INPUT})" ]; then
    /bin/sed -i '1s/^/%RWF\n/' ${INPUT}
fi

# update input file to use temporary storage, split into 500GB files
/bin/sed -i -E "s|s|%RWF(.*)|%RWF=${TMP}/1.rwf,200GB,${TMP2}/2.rwf,500GB,${TMP2}/3.rwf,500GB,${TMP2}/4.rwf,500GB,${TMP2}/,-1|g"
${TMP}/${INPUT}

. /apps/Gaussian/09D/g09/bsd/g09.profile

/app/Gaussian/09D/g09/g09 ${INPUT} > output.log

# job done, copy everything except .rwf back
echo "Copying from ${TMP}/ to ${PBS_O_WORKDIR}/"
/usr/bin/rsync -vax --exclude=*.rwf ${TMP}/ "${PBS_O_WORKDIR}/"

# delete my temporary files
[ $? -eq 0 ] && /bin/rm -rf ${TMP}

```

This script also requires that the input file contains a line starting with **%RWF**. This is so that the script can update the input file to specify that only the first part of the RWF be written to the compute node's local scratch space.

pisofOAM

pisofOAM generates a lot of output, not all of which is useful. In this example we use **crontab** to schedule the deletion of unwanted output while the job runs. Requests 3 week walltime.

```
#!/bin/bash
#PBS -l select=1:ncpus=8:mpiprocs=8:large-scratch=true
#PBS -l walltime=504:00:00
#PBS -m be

# make sure I'm the only one that can read my output
umask 0077
# create a temporary directory in /scratch-small-local
TMP=/scratch-small-local/${PBS_JOBID}
/bin/mkdir ${TMP}
echo "Temporary work dir: ${TMP}"

if [ ! -d "${TMP}" ]; then
    echo "Cannot create temporary directory. Disk probably full."
    exit 1
fi

# copy the input files to ${TMP}
echo "Copying from ${PBS_O_WORKDIR}/ to ${TMP}/"
/usr/bin/rsync -vax "${PBS_O_WORKDIR}/" ${TMP}/

cd ${TMP}

# start crontab, delete unwanted files every 6 hours
/bin/echo "0 */6 * * * /bin/find ${TMP} -regextype posix-egrep -regex '(${TMP}/processor[0-9]+)/((^[^/]*)|((uniform/.*)|ddt.*|phi.*|. *_0.*))' -exec rm {} \;" | /usr/bin/crontab

# Automatically calculate the number of processors
np=$(wc -l < ${PBS_NODEFILE})

module load compilers/gcc-4.8.2
module load openmpi/1.6.5

export MPI_BUFFER_SIZE=200000000

export FOAM_INST_DIR=/apps/OpenFOAM
foamDotFile=${FOAM_INST_DIR}/OpenFOAM-2.2.2/etc/bashrc
[ -f ${foamDotFile} ] && . ${foamDotFile}

blockMesh
decomposePar

mpirun -np ${np} pisoFoam -parallel > ${PBS_O_WORKDIR}/output.log

# remove crontab entry (assumes I only have one on this node)
/usr/bin/crontab -r

# job done, copy everything back
echo "Copying from ${TMP}/ to ${PBS_O_WORKDIR}/"
/usr/bin/rsync -vax --exclude "*_0.gz" --exclude "phi*.gz" --exclude "ddt*.gz" ${TMP}/ "${PBS_O_WORKDIR}/"

# delete my temporary files
[ $? -eq 0 ] && /bin/rm -rf ${TMP}
```

MSC Marc

Marc script requesting 8 cores, on 1 node, -m selects to mail end message and -M is the email address to send to.

```
#!/bin/bash
#PBS -N JobName
#PBS -l select=1:ncpus=8:mpiprocs=8
#PBS -l walltime=24:00:00
#PBS -l license_marc=8
#PBS -m e

INPUT=inputfile
```

```
# make sure I'm the only one that can read my output
umask 0077
```

```

TMP=/scratch-small-local/${PBS_JOBID}
mkdir -p ${TMP}

if [ ! -d "${TMP}" ]; then
    echo "Cannot create temporary directory. Disk probably full."
    exit 1
fi

# copy the input files to ${TMP}
echo "Copying from ${PBS_O_WORKDIR}/ to ${TMP}/"
/usr/bin/rsync -vax "${PBS_O_WORKDIR}"/ ${TMP}/

cd ${TMP}

module load app/marc

# get number of processors assigned
NPS=$(wc -l < ${PBS_NODEFILE})
HOSTS=hosts.${PBS_JOBID}

[ -f ${HOSTS} ] && /bin/rm ${HOSTS}
# create hosts file
uniq -c ${PBS_NODEFILE} | while read np host; do
    /bin/echo "${host} ${np}" >> ${HOSTS}
done

if [ ${NPS} -gt 1 ]; then
    run_marc -j ${INPUT} -ver n -back n -ci n -cr n -nps ${NPS} -host ${HOSTS}
else
    run_marc -j ${INPUT} -ver n -back n -ci n -cr n
fi

# job done, copy everything back
echo "Copying from ${TMP}/ to ${PBS_O_WORKDIR}/"
/usr/bin/rsync -vax ${TMP}/ "${PBS_O_WORKDIR}/"

# delete my temporary files
[ $? -eq 0 ] && /bin/rm -rf ${TMP}

```

mothur

mothur has massive data volumes, and therefore has to use local scratch space to avoid killing the file server. Requests 1 core on 1 node.

mothur's input can either be a file with all the commands to process listed, or the commands can be given on the commandline if prefixed with a #.

```

#!/bin/bash

#PBS -l select=1:ncpus=1:mpiprocs=1
#PBS -l walltime=24:00:00
#PBS -m e

# make sure I'm the only one that can read my output
umask 0077
TMP=/scratch-small-local/${PBS_JOBID}
mkdir -p ${TMP}

if [ ! -d "${TMP}" ]; then
    echo "Cannot create temporary directory. Disk probably full."
    exit 1
fi

# copy the input files to ${TMP}
echo "Copying from ${PBS_O_WORKDIR}/ to ${TMP}/"
/usr/bin/rsync -vax "${PBS_O_WORKDIR}"/ ${TMP}/

cd ${TMP}

module load app/mothur

# Automatically calculate the number of processors
np=$(wc -l < ${PBS_NODEFILE})

mothur inputfile

# could also put the commands on the command line
#mothur "#cluster.split(column=file.dist, name=file.names, Large=T, processors=${np})"

# job done, copy everything back

```

```
echo "Copying from ${TMP}/ to ${PBS_O_WORKDIR}/"
/usr/bin/rsync -vax ${TMP}/ "${PBS_O_WORKDIR}/"
```

```
# delete my temporary files
[ $? -eq 0 ] && /bin/rm -rf ${TMP}
```

Altair FEKO

```
#!/bin/bash

#PBS -l select=1:ncpus=10:mpiprocs=10:scratch=true:mem=100GB
#PBS -l walltime=48:00:00
#PBS -m e

INPUT=input.cfx

# make sure I'm the only one that can read my output
umask 0077
TMP=/scratch-small-local/${PBS_JOBID}
mkdir -p ${TMP}

if [ ! -d "${TMP}" ]; then
    echo "Cannot create temporary directory. Disk probably full."
    exit 1
fi

. /apps/altair/hyperworks/2017/altair/feko/bin/initfeko
# override job directory
export FEKO_USER_HOME=${PBS_O_WORKDIR}
# override temporary directory
export FEKO_TMPDIR=${TMP}
# limit to the 100GB requested
export FEKO_MAXALLOCM=$((1024 * 100))

runfeko ${INPUT} --use-job-scheduler

# delete my temporary files
[ $? -eq 0 ] && /bin/rm -rf ${TMP}
```

Numeca

This script assumes you have 4 Numeca files available for your project. If your project is named *proj1*, the required files are *proj1.iec*, *proj1.igg*, *proj1.bcs* and *proj1.cgns*.

Script requests 4 hours walltime, and uses 8 cores.

```
#!/bin/bash

#PBS -N proj1
#PBS -l select=1:ncpus=8:mpiprocs=8
#PBS -l walltime=04:00:00

INPUT=proj1

# make sure I'm the only one that can read my output
umask 0077
# create a temporary directory with the job id as name in /scratch-small-local
TMP=/scratch-small-local/${PBS_JOBID}
mkdir -p ${TMP}
echo "Temporary work dir: ${TMP}"

# copy the input files to ${TMP}
echo "Copying from ${PBS_O_WORKDIR}/ to ${TMP}/"
/usr/bin/rsync -vax "${PBS_O_WORKDIR}/" ${TMP}/

cd ${TMP}

NUMECA=/apps/numeca/bin
VERSION=90_3

# create hosts list
TMPH=$(/bin/mktemp)
/usr/bin/tail -n +2 ${PBS_NODEFILE} | /usr/bin/uniq -c | while read np host; do
    /bin/echo "${host} ${np}" >> ${TMPH}
done
NHOSTS=$(wc -l < ${TMPH})
LHOSTS=$(while read line; do echo -n ${line}; done < ${TMPH})
```

```

/bin/rm ${TMP}
# Create .run file
${NUMECA}/fine -nversion ${VERSION} -batch ${INPUT}.iec ${INPUT}.igg ${PBS_JOBID}.run
# Set up parallel run
${NUMECA}/fine -nversion ${VERSION} -batch -parallel ${PBS_JOBID}.run ${NHOSTS} ${LHOSTS}
# Start solver
${NUMECA}/euranusTurbo_parallel ${PBS_JOBID}.run -steering ${PBS_JOBID}.steering -nversion ${VERSION} -p4pg ${PBS_JOBID}.p4pg
# job done, copy everything back
echo "Copying from ${TMP}/ to ${PBS_O_WORKDIR}/"
/usr/bin/rsync -vax ${TMP}/ "${PBS_O_WORKDIR}/"
# delete my temporary files
[ $? -eq 0 ] && /bin/rm -rf ${TMP}

```

This script assumes you have 3 Numeca files available for your project. If your project is named *proj1*, the required files are *proj1.iec*, *proj1.trb* and *proj1.geomTurbo*.

Script requests 4 hours walltime, and uses 8 cores.

```

#!/bin/bash
#PBS -N proj1
#PBS -l select=1:ncpus=8:mpiprocs=8
#PBS -l walltime=04:00:00

INPUT=proj1

# make sure I'm the only one that can read my output
umask 0077
# create a temporary directory with the job id as name in /scratch-small-local
TMP=/scratch-small-local/${PBS_JOBID}
mkdir -p ${TMP}
echo "Temporary work dir: ${TMP}"

# copy the input files to ${TMP}
echo "Copying from ${PBS_O_WORKDIR}/ to ${TMP}/"
/usr/bin/rsync -vax "${PBS_O_WORKDIR}/" ${TMP}/

# Automatically calculate the number of processors - 1 less than requested, used for master process
NP=$(( (wc -l < ${PBS_NODEFILE}) - 1 ))

cd ${TMP}

NUMECA=/apps/numeca/bin
VERSION=101

# inputs .trb, .geomTurbo
# outputs .bcs, .cgns, .igg
/usr/bin/xvfb-run -d ${NUMECA}/igg -print -batch -nversion ${VERSION} -autogrid5 -trb ${TMP}/${INPUT}.trb -geomTurbo
${TMP}/${INPUT}.geomTurbo -mesh ${TMP}/${PBS_JOBID}.igg

# inputs .iec, .igg
# outputs .run
/usr/bin/xvfb-run -d ${NUMECA}/fine -print -batch -nversion ${VERSION} -project ${TMP}/${INPUT}.iec -mesh ${TMP}/${PBS_JOBID}.igg -
computation ${TMP}/${PBS_JOBID}.run

# inputs .run
# outputs .p4pg, .batch
/usr/bin/xvfb-run -d ${NUMECA}/fine -print -batch -nversion ${VERSION} -parallel -computation ${TMP}/${PBS_JOBID}.run -nproc ${NP} -
nbint 128 -nbreal 128

# inputs .run, .p4pg
${NUMECA}/euranusTurbo_parallel${VERSION} ${TMP}/${PBS_JOBID}.run -steering ${TMP}/${PBS_JOBID}.steering -p4pg ${TMP}/${PBS_JOBID}.p4pg

# job done, copy everything back
echo "Copying from ${TMP}/ to ${PBS_O_WORKDIR}/"
/usr/bin/rsync -vax ${TMP}/ "${PBS_O_WORKDIR}/"

```

```
# delete my temporary files
[ $? -eq 0 ] && /bin/rm -rf ${TMP}
```

Programs that handle job submission differently

MATLAB

Matlab R2024a can be used on the hpc cluster with a VPN connection. Usage description in this doc
Media:Getting_Started_With_Serial_And_Parallel_MATLAB_Stellenbosch.docx Download these scripts
Media:MATLAB_CONF.zip Enquire gerhardv@sun.ac.za

GPU

To use the GPU's specify the ngpus= parameter in a qsub -I for interactive session, or in the bash script - add it to the line that normally has the mem= and ncpus= parameters. Specify the queue with Qlist=ee. See below for an example for an interactive session. The mem= ncpus= values can be changed within reason. The bash script structure and other aspects of HPC use is described on <https://www0.sun.ac.za/hpc>

If you want to use the GPUs message gerhardv@sun.ac.za for inclusion on the GPU queue.

```
[gerhardv@hpc1 ~]$ qsub -I -l walltime=2:00:00 -l select=1:ncpus=4:mem=4GB:ngpus=1:Qlist=ee
qsub: waiting for job 253270.hpc1.hpc to start
qsub: job 253270.hpc1.hpc ready
[gerhardv@comp048 ~]$
```

The GPUs are commonly used via python with tensorflow or pytorch. Other GPU bindings are possible. Software sometimes automatically uses GPUs if they are present on the system, and R and MATLAB can also be configured to invoke GPU. In bash script form it would look something like this:

```
#!/bin/bash
#PBS -N JobName

#PBS -l select=1:ncpus=2:mem=16GB:ngpus=1:Qlist=ee
#PBS -l walltime=16:00:00
#PBS -m be
#PBS -e output.err
#PBS -o output.out
#PBS -M username@sun.ac.za

INPUT=inputfile

# make sure I'm the only one that can read my output
umask 0077
TMP=/scratch-small-local/${PBS_JOBID}
mkdir -p ${TMP}

if [ ! -d "${TMP}" ]; then
    echo "Cannot create temporary directory. Disk probably full."
    exit 1
fi

# copy the input files to ${TMP}
echo "Copying from ${PBS_O_WORKDIR}/ to ${TMP}/"
/usr/bin/rsync -vax "${PBS_O_WORKDIR}"/ "${TMP}/

cd ${TMP}

# choose version of python that has tensorflow/pytorch
module load python/3.8.1

python3 yourcode.py

# job done, copy everything back
echo "Copying from ${TMP}/ to ${PBS_O_WORKDIR}/"
/usr/bin/rsync -vax ${TMP}/ "${PBS_O_WORKDIR}/"

# delete my temporary files
[ $? -eq 0 ] && /bin/rm -rf ${TMP}
```

Typically the code would be developed offline on a notebook or with an interactive session and then benchmarked on the HPC with a bash script to optimise walltime and memory. The python code would then be run as a scheduler job using a bash script after development and debugging in interactive mode. Interactive session is launched as above with qsub -I.

```
[gerhardv@comp050 ~]$ module load python/3.8.1
[gerhardv@comp050 ~]$ python3
Python 3.8.1 (default, Jun 11 2021, 05:57:44)
[GCC 4.8.5 20150623 (Red Hat 4.8.5-44)] on linux
Type "help", "copyright", "credits" or "license()" for more information.
>>> import tensorflow as tf
>>> import torch as tt
>>>
```

There is a conda environment for tensorflow and python 3.9.13

```
[gerhardv@comp050 ~]$ module load app/tensorflow/2.9.2
(tf) [gerhardv@comp049 ~]$ python3
Python 3.9.13 (main, Aug 25 2022, 23:26:10)
[GCC 11.2.0] :: Anaconda, Inc. on linux
Type "help", "copyright", "credits" or "license()" for more information.
>>> import tensorflow as tf
2022-09-08 12:29:15.954105: I tensorflow/core/util/util.cc:169] oneDNN custom operations are on. You may see slightly different
numerical results due to floating-point round-off errors from different computation orders. To turn them off, set the environment
variable `TF_ENABLE_ONEDNN_OPTS=0`.
>>> print(tf.__version__)
2.9.2
>>>
>>> print(tf.config.list_physical_devices('GPU'))
[PhysicalDevice(name='/physical_device:GPU:0', device_type='GPU'), PhysicalDevice(name='/physical_device:GPU:1', device_type='GPU'),
PhysicalDevice(name='/physical_device:GPU:2', device_type='GPU')]
```

Retrieved from "https://www0.sun.ac.za/hpc/index.php?title=HOWTO_submit_jobs&oldid=869"

- This page was last edited on 24 October 2025, at 11:28.